The Complete Beginner's Guide to AI Engineering

From Zero to Understanding the Codebase Copilot

This guide assumes you know a little Python but nothing about AI. By the end, you will understand every concept used in this project — from scratch.

Table of Contents

1. What is Artificial Intelligence (in plain English)?
2. What is a Large Language Model (LLM)?
3. The Hallucination Problem
4. What is RAG? (Retrieval-Augmented Generation)
5. How Computers Understand Language — Vectors & Embeddings
6. What is a Vector Database?
7. How Search Works — Dense vs Sparse vs Hybrid
8. What is an AI Agent?
9. What is LangChain?
10. What is LangGraph? (Agent Orchestration)
11. What is a FastAPI Backend?
12. How All These Pieces Fit Together

---

1. What is Artificial Intelligence (in plain English)?

Artificial Intelligence (AI) is software that learns patterns from large amounts of data instead of being programmed with explicit rules.

The Old Way (Rule-Based):

```
IF the email contains the word "prize" THEN mark as spam
IF the email contains "free money" THEN mark as spam
```

This works until someone writes "pr!ze" or "fr33 money". You'd need to write millions of rules.

The AI Way (Pattern Learning): Feed the computer 10 million emails labeled "spam" or "not spam". Let the computer *discover its own patterns* by adjusting millions of internal numerical parameters (called **weights**) until it can classify new emails correctly.

This is called **Machine Learning**: learning from examples, not rules.

---

2. What is a Large Language Model (LLM)?

An LLM is an AI system trained on an enormous amount of text — books, websites, research papers, code repositories — using a special architecture called a **Transformer**.

Think of it as the world's most advanced autocomplete engine. After seeing "The capital of France is", it predicts "Paris" because that pattern appeared millions of times in training.

Famous LLMs:

- **OpenAI GPT-4** (powers ChatGPT)
- **Google Gemini** (what we use in this project)
- **Anthropic Claude** (our fallback)

How they work at a high level:

1. Your text is broken into small pieces called **tokens** (roughly one word each).
2. Each token is converted into a number vector.
3. The Transformer processes all tokens in parallel, letting each token "attend" to every other token (this is the famous **attention mechanism**).
4. The model predicts the most likely next token.
5. This repeats until a complete response is generated.

The key insight: The LLM has essentially compressed the knowledge of the entire internet into billions of numerical parameters (weights). It doesn't "know" facts like a database — it learned statistical associations.

---

3. The Hallucination Problem

Because LLMs predict the *statistically likely* next token, they will confidently generate text even when they have no relevant knowledge.

Example of hallucination:

> User: "What does the `authenticate_user` function in our `auth_service.py` do?"
> LLM: "The `authenticate_user` function checks the user's email and password against the database using bcrypt hashing..."
> Reality: The LLM made this up. It has never seen your private codebase.

This is called a **hallucination**: the model generates plausible-sounding but factually incorrect information.

For code systems, this is catastrophic — you might deploy a generated function that references APIs that don't exist.

---

4. What is RAG? (Retrieval-Augmented Generation)

RAG solves the hallucination problem by giving the LLM an "open book" right before it answers.

The core idea: Instead of asking the LLM to *remember* facts about your codebase, you:

1. **Retrieve** the relevant pieces of your codebase automatically.
2. **Augment** the prompt: paste those pieces directly into the prompt alongside the question.
3. **Generate** using only the provided text — the LLM is constrained to what it can see.

Analogy:

- **Closed-book exam** = Standard LLM (relies on memorized training data) → hallucinations.
- **Open-book exam** = RAG (LLM reads the relevant pages before answering) → accurate, cited answers.

A RAG prompt looks like this:

```
System: Answer using ONLY the provided code context. Cite file paths.

Context:
--- File: auth_service.py (Lines 12-25) ---
def authenticate_user(username: str, password: str) -> bool:
    user = db.query(User).filter(User.username == username).first()
    if not user:
        return False
    return bcrypt.checkpw(password.encode(), user.hashed_password)

Question: What does authenticate_user do?
```

Now the LLM can give an accurate, citation-backed answer because it literally has the code in front of it.

---

5. How Computers Understand Language — Vectors & Embeddings

Computers natively understand numbers, not words. To make language searchable and comparable, we convert text into **vectors** (lists of numbers).

What is a vector? A vector is just a list of numbers. For example:

- The word "king" might become: `[0.23, -0.71, 0.55, 0.02, ...]` (384 numbers)
- The word "queen" might become: `[0.21, -0.69, 0.52, 0.08, ...]` (very similar!)
- The word "pizza" might become: `[0.88, 0.12, -0.33, 0.91, ...]` (very different)

These are called **embeddings**. The distance between vectors represents semantic similarity.

How is similarity measured? We use **cosine similarity**: `cos(θ) = (A · B) / (|A| × |B|)`
Two vectors pointing in the same direction have cosine similarity ≈ 1 (very similar).
Two vectors pointing in opposite directions have cosine similarity ≈ -1 (very different).

The magic: An embedding model trained on billions of text examples learns that "multiply two numbers" and `def multiply(a, b): return a * b` should have similar vectors — even though they use completely different words!

In our project: We use the `all-MiniLM-L6-v2` model from HuggingFace. It converts any text into a 384-dimensional vector in milliseconds, running entirely locally on your CPU.

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vector = embeddings.embed_query("What does the multiply function do?")
# Returns a list of 384 floats: [0.12, -0.34, 0.78, ...]
```

---

6. What is a Vector Database?

A regular database stores text and lets you search by exact keywords. A **Vector Database** stores *embedding vectors* and lets you search by *semantic similarity*.

How it works:

1. You insert documents: each document is converted to a vector and stored alongside its metadata.
2. You query: your search string is converted to a vector.
3. The database finds stored vectors that are mathematically close to your query vector.
4. It returns those documents, ranked by similarity.

This is called **Approximate Nearest Neighbor (ANN) search** — finding the closest vectors in high-dimensional space very efficiently.

In our project: Qdrant

```
Qdrant is written in Rust, making it extremely fast.
It supports local file storage (no Docker needed for development).
Each "point" in Qdrant has:
  - id: a unique integer
  - vector: the embedding [0.23, -0.71, ...]
  - payload: metadata (file path, line numbers, code content)

# Inserting a code chunk into Qdrant
PointStruct(
    id=42,
    vector=[0.23, -0.71, 0.55, ...],  # 384-dim embedding
    payload={
        "file_path": "math_utils.py",
        "symbol_name": "multiply",
        "content": "def multiply(a, b):\n    return a * b"
    }
)
```

---

7. How Search Works — Dense vs Sparse vs Hybrid

Dense (Vector) Search

- **How**: Convert the query to a vector, find the nearest document vectors.
- **Good at**: Semantic concepts ("function that adds numbers" finds `def sum(a,b)`)
- **Bad at**: Exact keyword matching (`config_api_v2_timeout_ms` — hard to handle semantically)

Sparse (BM25 Keyword) Search

- **How**: Count word frequencies, use the classic TF-IDF formula to rank documents.
- **Good at**: Finding documents with exactly the words you searched for.
- **Bad at**: Synonyms and concepts ("fast" vs "quick", "error" vs "exception")

BM25 Formula:

```
BM25(q, d) = Σ IDF(word) × (tf × (k1+1)) / (tf + k1 × (1 − b + b × len(
```

Where `tf` = term frequency, `IDF` = inverse document frequency (rare words score higher), `k1` and `b` are tuning constants.

Hybrid Search (What We Use)

Run both strategies and merge the results using **Reciprocal Rank Fusion (RRF)**:

```
Score = 1/(60 + rank_in_dense_list) + 1/(60 + rank_in_sparse_list)
```

A document that appears near the top of *both* lists gets the highest combined score. This gives you the best of both worlds.

---

8. What is an AI Agent?

An **Agent** is an LLM that has been given:

1. A **specific role** (system prompt, e.g., "You are a Senior Code Reviewer")
2. **Tools** (optional functions it can call, like "search the database")
3. **A specific job to do** (e.g., review this PR diff)

Without the agent framework, every LLM call is stateless — you ask, it answers, done. An **Agent** can reason through multi-step problems, use tools, and even loop back to check its own work.

In our project, we have 4 agents:

Agent	Role (System Prompt)	What it does
QAAgent	"Expert codebase assistant. Cite file paths."	Answers questions about the code
ReviewAgent	"Expert code reviewer. Check correctness & architecture."	Reviews PR diffs
TestAgent	"Senior SDET. Write pytest test suites."	Generates unit tests
DocsAgent	"Technical writer. Write markdown documentation."	Generates documentation

Each agent gets the same retrieved context but produces radically different outputs because of their specialized prompts.

---

9. What is LangChain?

LangChain is a Python library that provides:

1. **A unified interface** to different LLM providers (Gemini, Anthropic, OpenAI, etc.)
2. **LCEL (LangChain Expression Language)** — a `|` pipe syntax to chain components

Example without LangChain (verbose):

```
import google.generativeai as genai
genai.configure(api_key="...")
model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content(f"System: You are a helper\nUser: {qu
text = response.candidates[0].content.parts[0].text
```

Example with LangChain (clean and composable):

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatGoogleGenerativeAI(model="gemini-3.7-flash")
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("user", "{question}")
])
chain = prompt | llm | StrOutputParser()
answer = chain.invoke({"question": "What is 2+2?"})
```

The `|` (pipe) operator chains the components: prompt → llm → parser. This is called a **runnable sequence** and is the core abstraction in our codebase.

LangChain also provides:

- `.with_fallbacks([fallback_llm])` — automatic failover
- `max_retries=3` — exponential backoff
- `ChatPromptTemplate` — structured prompting with system/user message roles

---

10. What is LangGraph? (Agent Orchestration)

LangGraph extends LangChain to build **stateful, multi-step agent workflows** as explicit graphs (state machines).

Why not just use Python if/else?

```
# The naive way (brittle)
if "review" in user_message:
    call_review_agent()
elif "test" in user_message:
    call_test_agent()
```

This is fragile (what if they say "audit" instead of "review"?), not observable, and can't be easily extended.

LangGraph's approach:

1. Define a **State** (shared data dictionary that flows through the graph)
2. Define **Nodes** (Python functions that receive state and return updated state)
3. Define **Edges** (connections between nodes, including conditional branching)
4. **Compile** the graph into an executable workflow

Our graph flow:

```
AppState enters graph with:
  - input_text: "What does multiply do?"
  - intent: ""
  - final_output: ""

→ [Supervisor Node]
   classify_intent() is called
   AppState.intent = "qa"

→ [QA Node]  (because intent == "qa")
   run_qa() is called
   AppState.final_output = "The multiply function..."

→ [END]
   graph.invoke() returns the final AppState
```

LangGraph handles all the routing automatically. You define the rules once, and every future request follows the graph structure deterministically.

---

11. What is a FastAPI Backend?

FastAPI is a modern Python web framework for building APIs (Application Programming Interfaces).

What is an API? An API is a way for two programs to talk to each other. The Web UI (running in your browser) needs to communicate with our Python AI pipeline (running on your computer). They do this over HTTP:

```
Browser sends: POST /api/chat
               Body: {"message": "What does multiply do?"}

Server returns: {"response": "The multiply function...", "intent": "qa"
```

Why FastAPI?

- **Async**: Can handle multiple requests simultaneously.
- **Auto-docs**: Automatically generates an interactive API documentation page at `/docs`.
- **Type safety**: Uses Python type hints for validation.
- **Fast**: One of the fastest Python frameworks available.

Our API (`api/main.py`):

```
@app.post("/api/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    # Invoke the entire LangGraph pipeline
    initial_state = {"input_text": request.message, "intent": "", "fina
```

```
    result_state = supervisor.graph.invoke(initial_state)

    return ChatResponse(
        response=result_state["final_output"],
        intent=result_state["intent"]
    )
```

---

12. How All These Pieces Fit Together

```
YOU TYPE A MESSAGE IN THE BROWSER
              |
              | (HTTP POST to /api/chat)
             v
        [FastAPI Server]
        api/main.py
              |
              | (calls supervisor.graph.invoke())
             v
        [LangGraph Graph]
              |
        [Supervisor Node]
        classify_intent()
        Gemini 3.5 Flash reads your message
        Returns: intent = "qa"
              |
              | (conditional edge: intent == "qa")
             v
        [QA Agent Node]
        qa_agent.answer_question()
              |
              | Step 1: Hybrid Search
             v
        [HybridSearcher]
        a) Dense: embed query → search Qdrant → top 30 chunks
        b) Sparse: BM25 keyword search → top 30 chunks
        c) RRF: merge both lists → ranked top 30
              |
              | Step 2: Reranking
             v
        [Cross-Encoder Reranker]
        Scores all 30 (query, chunk) pairs jointly
        Returns top 5 most relevant chunks
              |
              | Step 3: Multi-Hop Expansion
             v
        [MultiHopExpander]
        Reads dependency_graph.json
        Adds callers/callees of the top 5 chunks
        Returns ~8 enriched chunks
              |
              | Step 4: Generation + Verification Loop
```

```
      v
[Gemini 3.7 Flash] → generates answer with citations
[Gemini 3.5 Flash] → verifies: PASS ✓
      |
      | Step 5: Return to graph
      v
[END Node]
Returns AppState with final_output
      |
      | (back to FastAPI)
      v
HTTP Response: {"response": "The multiply function...", "intent
      |
      | (back to Browser)
      v
JavaScript receives response
marked.js renders Markdown → pretty HTML
highlight.js adds code syntax highlighting
Intent badge "qa" appears above response
      |
      v
YOU SEE THE ANSWER ON SCREEN
```

Key Vocabulary Cheat Sheet

Term	Simple Definition
LLM	An AI trained on billions of texts that can predict/generate text
Token	A small chunk of text (roughly 1 word) that LLMs process
Hallucination	When an LLM confidently states something false
Embedding	A list of numbers that represents the meaning of text
Vector Database	A database that stores embeddings and can search by semantic similarity
RAG	Giving the LLM relevant documents as context before answering
BM25	A classic keyword-based search algorithm
RRF	A formula for merging multiple ranked search results
Cross-Encoder	A model that scores query + document pairs together for high accuracy
AST	A tree representation of source code structure
Tree-Sitter	A library that parses source code into an AST
Agent	An LLM with a specific role and (optionally) tools
LangChain	A Python library providing unified LLM interfaces and chain composition
LangGraph	A library for building stateful multi-agent workflows as graphs
FastAPI	A Python web framework for building HTTP APIs
Exponential Backoff	Waiting longer and longer between retries when a request fails

Provider Fallback	Automatically switching to a backup AI provider when the primary fails